

# Deep Reinforcement Learning applied to Forex Market: Deep Deterministic Policy Gradient Trading System

PIC2 - Master in Computer Science and Engineering  
Instituto Superior Técnico, Universidade de Lisboa

Vicente Aser Marques Rebelo Castillo Lorenzo — 92569\*  
[vicente.aser.lorenzo@tecnico.ulisboa.pt](mailto:vicente.aser.lorenzo@tecnico.ulisboa.pt)

Advisor: Rui Fuentecilla Maia Ferreira Neves

**Abstract** This research proposes an innovative solution to improve automated Forex trading systems using the Deep Deterministic Policy Gradient algorithm within a Deep Reinforcement Learning framework. Despite advances in Deep Reinforcement Learning applications in financial markets, integrating Deep Deterministic Policy Gradient with Recurrent Neural Networks, particularly Long-Short Term Memory networks, remains underexplored in Forex trading. This study addresses this gap by developing a trading system that incorporates the Deep Deterministic Policy Gradient algorithm, enriched with Long-Short Term Memory networks to handle sequential data and temporal dependencies. The proposed system integrates with the cTrader platform, enabling online learning and real-time strategy execution through a client-server architecture. The evaluation framework includes extensive backtesting using historical data, with performance evaluated against baseline strategies through metrics such as Net Profit, Sharpe Ratio, and Maximum Drawdown. The robustness of the system will be improved by implementing advanced feature engineering, normalisation, regularisation, and adaptive learning techniques. This research aims to deliver a robust and adaptive solution for automated Forex trading, with the potential for superior risk-adjusted returns and significant advancements over existing methodologies. Spanning from June 2024 to February 2025, this project seeks to fill existing research gaps and establish a new benchmark in automated Forex trading through the innovative application of Deep Reinforcement Learning technologies.

**Keywords** — Forex Trading, Neural Networks, Deep Reinforcement Learning, Deep Deterministic Policy Gradient

---

\*I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa (<https://nape.tecnico.ulisboa.pt/en/apoio-ao-estudante/documentos-importantes/regulamentos-da-universidade-de-lisboa/>).

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                               | <b>3</b>  |
| 1.1      | Motivation . . . . .                                              | 3         |
| 1.2      | Objectives . . . . .                                              | 3         |
| <b>2</b> | <b>Background</b>                                                 | <b>4</b>  |
| 2.1      | Financial Markets and Trading . . . . .                           | 4         |
| 2.1.1    | Forex Market: Characteristics and Significance . . . . .          | 4         |
| 2.1.2    | Forex Trading: Fundamental Concepts and Operation . . . . .       | 5         |
| 2.1.3    | Financial Analysis: Main Types and Formulas . . . . .             | 6         |
| 2.1.4    | Algorithmic and Quantitative Trading . . . . .                    | 7         |
| 2.2      | Neural Networks . . . . .                                         | 7         |
| 2.2.1    | Fundamentals and History . . . . .                                | 9         |
| 2.2.2    | Key Concepts and Feed-Forward . . . . .                           | 10        |
| 2.2.3    | Learning using Backpropagation . . . . .                          | 10        |
| 2.2.4    | Bias-Variance Trade-off and Model Generalisation . . . . .        | 11        |
| 2.2.5    | Universal Approximation Theorem . . . . .                         | 12        |
| 2.3      | Reinforcement Learning . . . . .                                  | 12        |
| 2.3.1    | Fundamentals and History . . . . .                                | 12        |
| 2.3.2    | Key Concepts and Agent-Environment Interaction . . . . .          | 12        |
| 2.3.3    | Value Functions and Q-Functions . . . . .                         | 13        |
| 2.3.4    | Methodological Variants and Learning Methods . . . . .            | 14        |
| 2.4      | Deep Reinforcement Learning . . . . .                             | 15        |
| 2.4.1    | Fundamentals and History . . . . .                                | 15        |
| 2.4.2    | Deep Value-Based RL Algorithms: Deep Q-Networks . . . . .         | 15        |
| 2.4.3    | Deep Policy-Based RL Algorithms: Deep Policy Gradients . . . . .  | 16        |
| <b>3</b> | <b>Related Work</b>                                               | <b>16</b> |
| 3.1      | Survey Research: Identifying the State-of-the-Art . . . . .       | 16        |
| 3.2      | Article Research: Summarising Different Implementations . . . . . | 17        |
| 3.2.1    | Forex Market Implementations . . . . .                            | 17        |
| 3.2.2    | Stock Market Implementations . . . . .                            | 18        |
| 3.2.3    | Other Market Implementations . . . . .                            | 19        |
| 3.2.4    | Key Findings . . . . .                                            | 19        |
| <b>4</b> | <b>Proposed Solution</b>                                          | <b>20</b> |
| 4.1      | Research Gaps . . . . .                                           | 20        |
| 4.2      | Proposed Solution . . . . .                                       | 20        |
| 4.2.1    | Model Innovation . . . . .                                        | 20        |
| 4.2.2    | Training Innovation . . . . .                                     | 20        |
| 4.2.3    | Optimization Innovation . . . . .                                 | 23        |
| 4.2.4    | System Architecture . . . . .                                     | 23        |
| <b>5</b> | <b>Experimental Results</b>                                       | <b>24</b> |
| <b>6</b> | <b>Work Schedule</b>                                              | <b>25</b> |
| <b>7</b> | <b>Conclusion</b>                                                 | <b>25</b> |
|          | <b>Bibliography</b>                                               | <b>26</b> |

# 1 Introduction

Deep Reinforcement Learning (DRL) has significantly transformed the landscape of automated trading systems, offering sophisticated tools for decision-making in complex environments such as the Forex market. This research addresses a critical gap in the application of DRL to Forex trading by proposing a novel integration of the Deep Deterministic Policy Gradient (DDPG) algorithm with Recurrent Neural Networks (RNN), specifically Long-Short Term Memory (LSTM) networks.

## 1.1 Motivation

Automated Forex trading systems traditionally rely on simpler algorithms that often fail to capture the complex and dynamic sequential nature of financial markets. The challenge lies in developing a system that can not only handle the complexities of Forex trading, but also adapt to its dynamic environment in real-time. This research aims to improve automated Forex trading systems by implementing a DDPG algorithm, initially with a Fully Connected Neural Network (FNN) and potentially extending to LSTM networks for better handling of temporal dependencies.

The Forex market, being one of the most liquid and volatile markets, presents unique challenges and opportunities for automated trading systems. A robust DRL-based system that effectively navigates these challenges could significantly improve trading performance and risk management. This research is particularly important as it explores underutilised areas such as the application of DDPG with LSTM networks in Forex trading, and the integration of online learning capabilities through a novel client-server framework with the cTrader [1] platform. These advances have the potential to set new benchmarks in automated trading efficiency and adaptability.

Developing an effective DRL-based trading system is inherently difficult due to the complex nature of financial markets. Naive approaches often fail due to their inability to handle the high-dimensional and sequential nature of market data, leading to poor generalisation and performance. Additionally, achieving a balance between exploration and exploitation, as well as managing bias-variance trade-offs, are significant challenges that require advanced techniques such as feature engineering, normalisation, regularisation, and adaptive learning rates when optimising the models.

Although there have been significant advances in DRL, the integration of DDPG with LSTM networks specifically for Forex trading remains underexplored. Previous solutions often focus on simpler algorithms or different markets, lacking the sophistication to handle the complexities of the Forex market. Additionally, the implementation of online learning and real-time data processing has been limited by technological constraints and the complexity of developing robust client-server architectures. This research differentiates itself by addressing these gaps, leveraging advanced DRL techniques, and integrating with the cTrader platform to facilitate continuous learning and adaptation.

## 1.2 Objectives

This research proposes a comprehensive trading system comprising several key components:

- **DDPG Algorithm Implementation:** The system will initially implement DDPG with an FNN and extend to LSTM networks if feasible to improve the handling of sequential data.
- **Advanced Feature Engineering:** To improve model performance, advanced techniques such as feature engineering, normalisation, and technical analysis will be used.
- **Advanced Optimisation Techniques:** Techniques like regularisation,  $\epsilon$ -greedy policies, and Gaussian noise will be used to balance bias/variance and exploration/exploitation problems.
- **cTrader Platform Integration:** A novel client-server framework will be developed that will enable the system to process real-time data and execute strategies with online learning capabilities.

The proposed timeline from June 2024 to February 2025 ensures a systematic approach to development, testing, and evaluation, with the aim of providing a robust, adaptive, and efficient solution for

automated Forex trading. This study is a literature extension of foundational work in DRL, including key contributions such as [2], who demonstrated the potential of deep learning to master Atari games through reinforcement learning with raw pixel input; [3], who introduced the Deep Q-Network (DQN) algorithm that exceeded human performance in Atari games; [4], who improved DQN by addressing its overestimation bias through the Double DQN (DDQN) algorithm; and [5], who first implemented DDPG, demonstrating its efficacy in solving tasks with continuous action spaces. This research also draws upon the fundamental knowledge presented in key texts such as [6] on deep learning and [7] on reinforcement learning. This research aims to establish a new benchmark in automated Forex trading, advancing the field through innovative applications of DRL technologies and addressing significant existing gaps in the literature.

## 2 Background

In this section, we discuss the topics that are crucial to understanding the complexities and scope of our research. We begin by exploring the core principles of financial markets with an emphasis on the Forex market. Next, we explore two of the great fields in Machine Learning, Neural Networks and Reinforcement Learning. Lastly, we investigate the intersection of these two branches through an exploration of Deep Reinforcement Learning and how this innovative field can be used for Forex trading.

### 2.1 Financial Markets and Trading

Financial markets play a crucial role in the global economy by facilitating capital flow, ensuring liquidity, and managing risks. They provide a platform for entities to trade assets, hedge financial risks, and fund their operations. Various kinds of markets exist, each with its unique function. Table 1 provides a summary of the primary financial markets and the roles they play in the economy. Although each market serves a distinct purpose, they are interconnected in such a way that actions in one market can influence others, highlighting their significance. The behaviour of these markets is influenced by a range of participants, including investors and major financial institutions whose actions can influence asset prices and market trends. In addition, regulatory bodies play an important role in maintaining the integrity of these markets by enforcing policies that protect investors and contribute to prosperity. Technological advances, such as trading platforms and algorithmic strategies, have made the markets more accessible and sophisticated. However, they also bring challenges such as increased volatility and regulatory uncertainty [8].

| Market                | Economic Role                                              | Asset Examples                 |
|-----------------------|------------------------------------------------------------|--------------------------------|
| Stock Market          | Capital raising through company shares trading.            | Stocks, Equities               |
| Bond Market           | Debt financing by entities borrowing from investors.       | Government and Corporate Bonds |
| Commodities Market    | Trading of essential raw materials and agricultural goods. | Oil, Gold, Wheat               |
| Forex Market          | Currency exchange for international trade.                 | USD, EUR, JPY, GBP             |
| Derivatives Market    | Trading contracts for hedging or speculation.              | Futures, Options, Swaps        |
| Indices Market        | Aggregating and tracking market or sector performance.     | S&P 500, NASDAQ                |
| Cryptocurrency Market | Exchanging digital currencies.                             | Bitcoin, Ethereum              |

**Table 1:** Summary of Financial Markets and respective Economic Role [8].

#### 2.1.1 Forex Market: Characteristics and Significance

The Foreign Exchange market, or the Forex market, is a marketplace where currencies are exchanged. It is the world’s most liquid market, operating continuously across different time zones due to the involvement of banks and institutions worldwide. Major currency pairs such as EUR/USD and GBP/USD play

an important role in this market, as they are the most traded pairs and offer information on geopolitical trends. Forex trading is influenced by many factors such as data availability, political events, interest rates, and the use of leverage. Leverage allows investors to control significant positions with minimal capital, amplifying both potential profits and losses. The unique characteristics of Forex, including its liquidity, diverse participants, and sensitivity to events, make it an attractive topic for advanced financial research and algorithmic strategies. However, given its complex, noisy, and volatile nature, advanced predictive models are required to be able to exploit this type of market, demanding adaptive and intelligence strategies, enhancing the importance of machine learning usage [9].

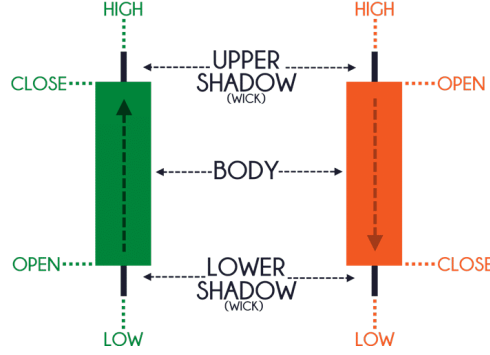
### 2.1.2 Forex Trading: Fundamental Concepts and Operation

Trading currency pairs involves buying one currency and simultaneously selling another. For instance, when we buy the EUR/USD pair we are essentially purchasing Euros and selling US Dollars based on our prediction that the Euro will strengthen against the US Dollar. In this research, we will be focussing on major currency pairs: EUR/USD, USD/JPY, GBP/USD, AUD/USD, USD/CHF, NZD/USD and USD/CAD. These pairs refer to recognised currencies, mainly European Euro (EUR), US Dollar (USD), Japanese Yen (JPY), Great Britain Pound (GBP), Australian Dollar (AUD), Swiss Franc (CHF) and New Zealand Dollar (NZD). In this way, understanding the value of these pairs offers valuable information on global economic trends, and deciding whether to buy or sell a pair is based on predicting the strength of one currency compared to another [9].

Forex trading occurs in a decentralised market that operates globally. This means that there is no exchange where all transactions take place. Instead, a network of market makers primarily composed of banks (known as the interbank market) and brokers facilitate trades. Interbank transactions involve trades with large volumes, and they are usually dominated by institutions such as multinational corporations and major hedge funds because of their transaction sizes. However, retail traders and small investors can participate in this market through brokers who have access to the interbank network. These brokers serve as intermediaries, facilitating trading for investors by connecting them to the larger currency exchange network. Both market makers and brokers are always ready to buy or sell currencies at quoted prices. The way they operate is by providing two prices publicly: the bid price (the price at which they are willing to buy the pair) and the ask price (the price at which they are willing to sell it). The difference between these two prices, known as the spread, serves as a commission for market makers and brokers. The absence of a central trading venue might suggest a lack of regulation, however, the Forex market is self-regulated by competition among market-makers. This competitive environment ensures that pricing remains fair and spreads are kept tight, thus preserving market integrity and transparency for all participants [9].

To understand the Forex market, it is important to understand how traders interpret market movements. Candlestick charts are a popular approach in this regard as they provide a view of price fluctuations. Each chart is made up of candlesticks, which are structures that represent price movements over a time span that can range from minutes to years. Each individual price movement is known as a tick and occurs when buyers and sellers agree on a price, leading to adjustments in bid and ask prices in the order book. Each candlestick, named for its resemblance to a double-sided candle, provides information on the opening price (O), the highest price (H), the lowest price (L), and the closing price (C) for a specific period of time (see Figure 1). The colour of the candlestick represents the price movement, with lighter shades indicating an increase and darker shades indicating a decrease. This allows traders to gain a fast understanding of the direction of the market just by looking at the chart. Although candlestick charts show price movements, they do not directly show the trading volume (V), which is a measure of the number of ticks or transactions in a given time period, but it is often shown on the chart as an additional indicator of market momentum [10].

This research will use data containing opening price, highest price, lowest price, closing price and tick volume, or for short, OHLCV data, as it includes all main elements for analysing market behaviour [10]. It will also incorporate indicators that interpret this data to provide additional prediction power for the model, as we will see next.



**Figure 1:** Candlesticks encoding upward (bullish) and downward (bearish) movements respectively [11].

### 2.1.3 Financial Analysis: Main Types and Formulas

Now, we move from representing data in Forex trading to examining it in order to make informed trading decisions, focussing on methods that provide deeper insight into market movement and provide predictive power to the trader. Financial analysis is a complex domain, generally categorised into three main types [12]:

- **Fundamental Analysis:** examines economic factors that affect currency prices. This includes analysis of GDP growth, employment rates, interest rates, and monetary policies, which help predict currency trends based on a nation’s economic health. However, the qualitative nature of these data poses challenges to computational models that prefer structured numerical input.
- **Sentiment Analysis:** seeks to quantify the mood of the market by analysing news, market commentary, and similar qualitative data. It aims to measure how collective emotions impact market trends, although its complexity and need for advanced natural language processing make it less straightforward for model integration.
- **Technical Analysis:** involves analysing previous OHLCV data to predict future market movements. It operates on the principle that past market behaviour will repeat itself, and thus identifying past patterns or metrics on current data can predict future movements. It is preferred for its quantitative results, which can be used by complex machine learning models.

This research will focus on the use of technical analysis to complement OHLCV data. This type of analysis can be divided into indicators and candlestick patterns. Focussing on indicators, these are metrics created mainly by mathematicians and economists calculated based on historical market data and offer the trader a quantitative approach to understanding market dynamics and predicting future movements. They are generally categorised into four main types [12]:

- **Trend Indicators:** Used to identify the prevailing direction of the market by filtering out market noise. Refer to Table 2 for more details on frequently used indicators.
- **Momentum Indicators:** Used to capture the momentum of a given trend, whether it is strong or weak, and if a reversal is expected. Refer to Table 3 for more details on frequently used indicators.
- **Volume Indicators:** Used to provide a perspective on the trading volume, providing clues about the strength of a price movement. Refer to Table 4 for more details on frequently used indicators.
- **Volatility Indicators:** Used to identify the level of uncertainty or risk associated with the volatility of the price movement. Refer to Table 5 for more details on frequently used indicators.

| Indicator Name                    | Formula                                                                                                                                                                                                                                                                                                                                                          | Range |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Simple Moving Average             | $SMA(N)_t = \frac{1}{N} \sum_{i=1}^N Price_{t-N+i}$                                                                                                                                                                                                                                                                                                              | -     |
| Exponential Moving Average        | $EMA(N)_t = \frac{2}{N+1} \times Price_t + (1 - \frac{2}{N+1}) \times EMA(N)_{t-1}$                                                                                                                                                                                                                                                                              | -     |
| Weighted Moving Average           | $WMA(N)_t = \frac{2}{N \times (N+1)} \sum_{i=1}^N i \times Price_{t-N+i}$                                                                                                                                                                                                                                                                                        | -     |
| Double Exponential Moving Average | $DEMA(N)_t = 2 \times EMA(N)_t - EMA_2(N)_t$                                                                                                                                                                                                                                                                                                                     | -     |
| Triple Exponential Moving Average | $TEMA(N)_t = 3 \times [EMA(N)_t - EMA_2(N)_t] + EMA_3(N)_t$                                                                                                                                                                                                                                                                                                      | -     |
| Hull Moving Average               | $HMA(N)_t = WMA \left( 2 \times WMA \left( \frac{N}{2} \right)_t - WMA(N)_t \right)$                                                                                                                                                                                                                                                                             | -     |
| Triangular Moving Average         | $TRIMA(N)_t = \frac{1}{N} \sum_{i=1}^N SMA(N)_{t-N+i}$                                                                                                                                                                                                                                                                                                           | -     |
| Kaufman Adaptive Moving Average   | $KAMA(N)_t = KAMA(N)_{t-1} + SC \times (Price_t - KAMA(N)_{t-1})$<br>$SC = [ER \times (\phi_{short} - \phi_{long}) + \phi_{long}]^2$ , $\phi_{short} = \frac{2}{N_{short}+1}$ , $\phi_{long} = \frac{2}{N_{long}+1}$<br>$ER = \frac{ Price_t - Price_{t-N} }{\sum_{i=1}^N  Price_{t-i+1} - Price_{t-i} }$<br>where typically $N_{short} = 2$ and $N_{long} = 30$ | -     |

**Table 2:** Summary of frequently used Trend Technical Indicators [13].

Focussing now on candlestick patterns, unlike indicators which are derived through mathematical calculations of market data, candlestick patterns offer a more visual approach to analysing market movements. These patterns represent price movements in a graphical format that can intuitively signal potential changes in market behaviour [10]. Although primarily qualitative, the visual signals provided by candlestick patterns can also be quantified and integrated into machine learning models, allowing for a comprehensive analysis that combines the depth of technical indicators with the intuitive insights of candlestick patterns [13]. This dual approach enriches the analytical framework used in predicting future market movements based on historical data.

### 2.1.4 Algorithmic and Quantitative Trading

Financial markets have evolved significantly, increasingly influenced by technology, leading to the predominance of algorithms in trading strategies. Algorithmic trading is a relatively new field of study, using computer programmes to execute trades much faster and in greater volumes than human traders, by leveraging the machines' ability to efficiently analyse large datasets, interpret market conditions, and execute trades based on set criteria. Used in pre-trade analysis, signal generation, and trade execution, algorithmic trading is essential for large institutions and hedge funds to implement complex strategies rapidly and on a large scale. Quantitative trading, driven by quants (quantitative analysts), is a more advanced subfield of algorithmic trading, based on mathematical and statistical models to find trading opportunities. It involves applying these models to market data to identify patterns and correlations, using historical data analysis and machine learning. This data-centric approach demands a strong grasp of finance and mathematics, focussing on systematic strategies over human judgment [14].

Both algorithmic and quantitative trading have transformed financial markets, enhancing efficiency and introducing new dynamics such as high-frequency trading (HFT). This mix of finance and technology has reshaped trading, highlighting the importance of advanced machine learning models, such as neural networks, for analysing complex market data and reinforcement learning for developing intelligent trading strategies [13].

## 2.2 Neural Networks

In today's world, where we have an abundance of data and easy access to it, the power of machine learning has become a game changer in many fields. At its core, machine learning relies on algorithms to identify patterns in data and to make decisions with minimal human intervention. Tom Mitchell, a leading figure in the field, explains it as [15]:

| Indicator Name                        | Formula                                                                                                                                                  | Range       |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Price Momentum                        | $PM_t = \frac{Price_t - Price_{t-1}}{Price_{t-1}}$                                                                                                       | -           |
| Price Acceleration                    | $PA_t = \frac{\text{Short Linear Regression Coefficient}}{\text{Long Linear Regression Coefficient}} = \frac{\beta_{\text{short}}}{\beta_{\text{long}}}$ | -           |
| Relative Strength Index               | $RSI(N)_t = 100 - \frac{100}{1 + \frac{\text{Average Gain}_t}{\text{Average Loss}_t}}$                                                                   | [0, 100]    |
| Aroon Oscillator                      | $\text{Average Gain}_t = \frac{1}{N} \sum_{i=1}^N \max(\text{Price}_i - \text{Price}_{i-1}, 0)$                                                          | [-100, 100] |
|                                       | $\text{Average Loss}_t = \frac{1}{N} \sum_{i=1}^N \max(\text{Price}_{i-1} - \text{Price}_i, 0)$                                                          |             |
|                                       | $\text{Aroon}(N)_t = \text{Aroon Up}(N)_t - \text{Aroon Down}(N)_t$                                                                                      |             |
|                                       | $\text{Aroon Up}(N)_t = \left( \frac{N - \text{Periods Since Highest High in } N}{N} \right) \times 100$                                                 |             |
|                                       | $\text{Aroon Down}(N)_t = \left( \frac{N - \text{Periods Since Lowest Low in } N}{N} \right) \times 100$                                                 |             |
| Balance of Power                      | $BOP_t = \frac{\text{Close}_t - \text{Open}_t}{\text{High}_t - \text{Low}_t}$                                                                            | [-1, 1]     |
| Commodity Channel Index               | $CCI(N)_t = \frac{TP_t - \text{SMA}_{TP}(N)_t}{0.015 \times \text{Mean Deviation}(N)_t}$                                                                 | -           |
|                                       | $TP_t = \frac{\text{High}_t + \text{Low}_t + \text{Close}_t}{3}$                                                                                         |             |
|                                       | $\text{Mean Deviation}(N)_t = \frac{1}{N} \sum_{i=1}^N  TP_i - \text{SMA}_{TP}(N)_t $                                                                    |             |
| Moving Average Convergence Divergence | $MACD_t = \text{EMA}(N_{\text{short}})_t - \text{EMA}(N_{\text{long}})_t$                                                                                | -           |
|                                       | where typically $N_{\text{short}} = 12$ and $N_{\text{long}} = 26$                                                                                       |             |
| Stochastic RSI                        | $\text{StochRSI}(N)_t = \frac{RSI(N)_t - \max(RSI(N))}{\max(RSI(N)) - \min(RSI(N))}$                                                                     | [0, 100]    |
| Stochastic Oscillator                 | $\text{Stoch \%K}(N)_t = \frac{\text{Close}_t - \text{LL}(N)_t}{\text{HH}(N)_t - \text{LL}(N)_t} \times 100$                                             | [0, 100]    |
|                                       | $\text{Stoch \%D}(M)_t = \text{SMA}_{\text{Stoch \%K}(N)}(M)_t$                                                                                          | [0, 100]    |
|                                       | $\text{HH}(N)_t = \max_{i=t-N+1}^t (\text{High}_i), \text{LL}(N)_t = \min_{i=t-N+1}^t (\text{Low}_i)$                                                    |             |
| Ultimate Oscillator                   | $UO_t = \frac{4 \times \text{Avg}_t(N_1) + 2 \times \text{Avg}_t(N_2) + \text{Avg}_t(N_3)}{4 + 2 + 1} \times 100$                                        | [0, 100]    |
|                                       | $\text{Avg}_t(N) = \frac{\sum_{i=0}^{N-1} BP_{t-i}}{\sum_{i=0}^{N-1} \text{TR}_{t-i}}$                                                                   |             |
|                                       | $BP_t = \text{Close}_t - \min(\text{Close}_{t-1}, \text{Low}_t)$                                                                                         |             |
|                                       | $\text{TR}_t = \max(\text{Close}_{t-1}, \text{High}_t) - \min(\text{Close}_{t-1}, \text{Low}_t)$                                                         |             |
|                                       | where typically $N_1 = 7, N_2 = 14$ , and $N_3 = 28$                                                                                                     |             |
| Williams %R                           | $\text{Williams \%R}(N)_t = \frac{\text{HH}(N)_t - \text{Close}_t}{\text{HH}(N)_t - \text{LL}(N)_t} \times -100$                                         | [-100, 0]   |

**Table 3:** Summary of frequently used Momentum Technical Indicators [13].

| Indicator Name                               | Formula                                                                                                                                                                                                                                                      | Range |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Chaikin Accumulation/Distribution Line       | $AD_t = AD_{t-1} + \text{MFV}_t$                                                                                                                                                                                                                             | -     |
|                                              | $\text{MFV}_t = \frac{\text{Close}_t - \text{Low}_t}{\text{High}_t - \text{Low}_t} \times \text{Volume}_t$                                                                                                                                                   |       |
| Chaikin Accumulation/Distribution Oscillator | $\text{ADOSC}_t = \text{EMA}_{AD}(N_{\text{short}})_t - \text{EMA}_{AD}(N_{\text{long}})_t$                                                                                                                                                                  | -     |
|                                              | where typically $N_{\text{short}} = 3$ and $N_{\text{long}} = 10$                                                                                                                                                                                            |       |
| On-Balance Volume                            | $\text{OBV}_t = \begin{cases} \text{OBV}_{t-1} + \text{Volume}_t & \text{if } \text{Price}_t > \text{Price}_{t-1} \\ \text{OBV}_{t-1} - \text{Volume}_t & \text{if } \text{Price}_t < \text{Price}_{t-1} \\ \text{OBV}_{t-1} & \text{otherwise} \end{cases}$ | -     |

**Table 4:** Summary of frequently used Volume Technical Indicators [13].

"A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with



| Indicator Name                | Formula                                                                                                                                                                                    | Range |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Standard Deviation            | $SD(N)_t = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (\text{Price}_i - \overline{\text{Price}})^2}$                                                                                                 | -     |
| Bollinger Bands               | Upper $BB(N, k)_t = SMA(N)_t + k \times SD(N)_t$                                                                                                                                           | -     |
|                               | Lower $BB(N, k)_t = SMA(N)_t - k \times SD(N)_t$                                                                                                                                           |       |
|                               | where typically $k = 2$                                                                                                                                                                    |       |
| Average True Range            | $ATR(N)_t = \frac{1}{N} \sum_{i=1}^N \text{TRANGE}_i$<br>$\text{TRANGE}_t = \max(\text{High}_t - \text{Low}_t,  \text{High}_t - \text{Close}_{t-1} ,  \text{Low}_t - \text{Close}_{t-1} )$ | -     |
| Normalised Average True Range | $NATR(N)_t = \frac{ATR(N)_t}{\text{Close}_t} \times 100$                                                                                                                                   | -     |
| Chaikin Volatility Index      | $CVI(N)_t = \frac{1}{N} \sum_{i=1}^N (\text{High}_i - \text{Low}_i)$                                                                                                                       | -     |

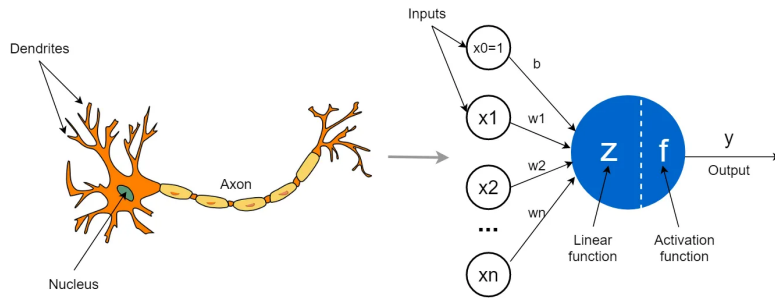
**Table 5:** Summary of frequently used Volatility Technical Indicators [13].

experience E."

This captures the essence of machine learning: creating models that automatically improve through experience. Neural networks, the topic of this section, are a class of machine learning models that operate within this framework by simulating the complex networks of the human brain.

### 2.2.1 Fundamentals and History

Neural Networks (NN), named for their similarity to the structure of the human brain, are algorithms designed for pattern recognition. In the human brain, a neuron receives signals through structures called dendrites, processes these signals in the nucleus, and passes the resultant processed signal down its axon to the dendrites of adjacent neurons. Similarly, in a neural network, each node represents an artificial neuron that receives input, processes it through a mathematical function, and passes the output to the subsequent layer of nodes. In both cases, the strength and nature of the connections between neurons determine the behaviour of the network [6]. Refer to Figure 2 for a visual representation of the concepts explained above.



**Figure 2:** Human Neuron compared to an Artificial Neuron [16].

The concept of neural networks dates back to the 1950s with the perceptron, a simple model of a biological neuron developed by Frank Rosenblatt [17]. The perceptron marked the early stages of NN research, but was limited to linearly separable datasets, leading to some periods of reduced interest in the field, known as "AI winters." However, the 1980s brought a significant breakthrough with the backpropagation algorithm, enabling neural networks to learn patterns in non-linearly separable datasets using complex and multilayered models [18].

### 2.2.2 Key Concepts and Feed-Forward

In a neural network, the feed-forward computation within each layer  $l$  is based on the data processed by its predecessor. For the first layer, this input  $x$  is the raw data itself, while for subsequent layers, it is the output of the previous layer  $h^{(l-1)}(x)$ . The computation in each layer can be formalised as [6]:

$$z^{(l)}(x) = W^{(l)}h^{(l-1)}(x) + b^{(l)} \quad (1)$$

Here, the weight matrix  $W^{(l)} \in \mathbb{R}^{K_l \times K_{l-1}}$  and the bias vector  $b^{(l)} \in \mathbb{R}^{K_l}$  are required for the computation at layer  $l$ , with  $K_l$  representing the number of neurons or units in layer  $l$ , and  $K_{l-1}$  indicating the number of neurons in the preceding layer  $l-1$ . In particular,  $W^{(1)} \in \mathbb{R}^{K_1 \times D}$  for the first layer, where  $D$  is the size of the input data. The activation function  $g$  is subsequently applied to the pre-activation  $z^{(l)}(x) \in \mathbb{R}^{K_l}$  to produce the activated output  $h^{(l)}(x) \in \mathbb{R}^{K_l}$  [6]:

$$h^{(l)}(x) = g(z^{(l)}(x)) \quad (2)$$

During the design of a neural network, the choice of activation functions is very important. Starting by the hidden layers, we may employ a variety of non-linear activation functions, adding layers of expressiveness to the network and allowing it to learn more complex patterns. Finally, the activation function used in the output layer is determined by the type of task the network is going to perform. This careful selection directly influences the network's ability to accurately and effectively model the dataset [6]. Refer to Table 6 for a summary of the most common activation functions.

| Activation Function | Formula                                                      | Description                                                      |
|---------------------|--------------------------------------------------------------|------------------------------------------------------------------|
| Linear              | $a(x) = x$                                                   | Suitable for regression tasks.                                   |
| Sigmoid             | $\sigma(x) = \frac{1}{1+e^{-x}}$                             | Maps inputs to (0, 1), for binary classification.                |
| Hyperbolic Tangent  | $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$               | Maps inputs to (-1, 1), offering richer gradients than sigmoid.  |
| ReLU                | $\text{ReLU}(x) = \max(0, x)$                                | Encourages sparse activation, helps with gradient flow.          |
| Softmax             | $\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}}$ | Turns logits into probabilities, for multi-class classification. |

**Table 6:** Summary of the most common Activation Functions [6].

### 2.2.3 Learning using Backpropagation

Neural networks learn by iteratively adjusting weights and biases to minimise the error between predicted outputs and actual target values. This learning process is guided by a loss function that quantifies the model's error. The goal is to optimise the network by finding parameters that minimise this loss function. Different problems require different loss functions, such as mean squared error for regression tasks and cross-entropy for classification [6]. Refer to Table 7 for a detailed overview of their formulas.

| Loss Function                           | Formula                                        | Application    |
|-----------------------------------------|------------------------------------------------|----------------|
| Mean Squared Error (MSE)                | $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$ | Regression     |
| Negative Log-Likelihood (Cross-Entropy) | $-\sum_{i=1}^n y_i \log(\hat{y}_i)$            | Classification |

**Table 7:** Summary of the most common Loss Functions [6].

The core algorithm used to train neural networks is known as backpropagation [18]. It involves calculating the gradients of the loss function relative to the network's weights and biases, which mathematically indicate the steepest increase in the loss function. Refer to Algorithm 1 for a detailed overview of how to calculate these gradients. Then, gradient descent algorithm is used to iteratively update the

weights and biases in the opposite direction of the gradients, thus reducing the loss and hopefully increasing the network's accuracy. Its variants, outlined with the respective formulas in Table 8, address different data and computational needs. Batch gradient descent uses the entire dataset for updates, ensuring completeness but potentially struggling with large datasets. Stochastic gradient descent updates the parameters per data point, offering speed at the cost of stability. Mini-batch gradient descent strikes a balance, using data subsets for more stable and computationally efficient updates [6].

---

**Algorithm 1** Gradient of the Loss Function [6].

---

**Require:**  $f$ : network function,  $\theta$ : network parameters,  $x$ : input data,  $y$ : true label,  $l$ : number of layers,  $L$ : loss function  
 Compute the output of the network  $\hat{y} = f(x; \theta)$   
 Compute the gradient of the loss function with respect to the output layer before activation:  
 $\nabla_{z^{(l+1)}} L(f(x; \theta), y) = \frac{\partial L(f(x; \theta), y)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(l+1)}}$   
**for**  $\ell = l + 1$  to 1 **do**  
   Compute gradients of hidden layer parameters:  
    $\nabla_{W^{(\ell)}} L(f(x; \theta), y) = \nabla_{z^{(\ell)}} L(f(x; \theta), y) \cdot h^{(\ell-1)T}$   
    $\nabla_{b^{(\ell)}} L(f(x; \theta), y) = \nabla_{z^{(\ell)}} L(f(x; \theta), y)$   
   Compute gradient of previous hidden layer:  
    $\nabla_{h^{(\ell-1)}} L(f(x; \theta), y) = W^{(\ell)T} \cdot \nabla_{z^{(\ell)}} L(f(x; \theta), y)$   
   Compute gradient of previous hidden layer (before activation):  
    $\nabla_{z^{(\ell-1)}} L(f(x; \theta), y) = \nabla_{h^{(\ell-1)}} L(f(x; \theta), y) \odot g'(z^{(\ell-1)})$   
**end for**

---

In Table 8, the learning rate  $\eta$  is an hyperparameter that influences the convergence of gradient descent. An optimal learning rate balances convergence speed and precision, preventing overshooting or slow progress that could get trapped in local minima. Adaptive learning rate techniques by adjusting  $\eta$  during training improve learning efficacy, emphasising the balance between convergence speed and accuracy in neural network training. Some other techniques to improve convergence and its speed involve normalisation and standardisation of the input data [6].

| Algorithm                   | Formula                                                                             | Description                              |
|-----------------------------|-------------------------------------------------------------------------------------|------------------------------------------|
| Batch Gradient Descent      | $\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; X, y)$                     | Updates parameters using all data        |
| Mini-Batch Gradient Descent | $\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; X^{(i:i+n)}, y^{(i:i+n)})$ | Updates parameters using subsets of data |
| Stochastic Gradient Descent | $\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}, y^{(i)})$         | Updates parameters per data point        |

---

**Table 8:** Summary of the most common Gradient Descent variants [6].

### 2.2.4 Bias-Variance Trade-off and Model Generalisation

The generalisation power of a neural network depends on finding a balance between bias and variance. Bias leads to underfitting, where the model overlooks underlying trends due to overly simplistic assumptions, while variance leads to overfitting, where the model interprets noise as a signal due to excessive complexity. Both underfitting and overfitting result in poor performance of the model on new data, a significant issue in Forex trading, where accurate market signal detection is essential. To prevent this, cross-validation is commonly employed to evaluate a model's generalisation power in independent training, validation and testing datasets. Regularisation techniques such as L1 and L2 are also used to moderate the model complexity by adding penalties to the loss function. Furthermore, principal component analysis (PCA) can help to find the right balance between bias and variance by selecting, extracting and generating relevant features to the model. Implementing these strategies increases the change of training a model with more generalisation power, thus allowing it to perform in out-of-sample data [6].

Although important, these techniques are not the main focus of this research and will not be explored in full detail.

### 2.2.5 Universal Approximation Theorem

The universal approximation theorem, formalised by Kurt Hornik in 1989, states [19]:

"A Neural Network with one hidden layer and a linear output can approximate arbitrarily well any continuous function, given enough hidden units."

This validates the capacity of neural networks to serve as universal function approximators, which is an important result in the context of creating predictive models for Forex trading because, in theory, they should be able to capture the complex underlying patterns that drive market movements, in conjunction with reinforcement learning, as we will see in the next sections.

## 2.3 Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning that focusses on an agent's ability to interact with its environment and make decisions. Unlike other types of learning methods, RL emphasises decision-making in potentially complex and uncertain environments. The goal of RL is to maximise rewards by learning from the consequences of actions, through trial and error. This approach has found applications in a range of fields such as robotics and finance, enabling the development of systems that improve their performance over time and adapt to new situations and make a series of decisions to achieve desired outcomes.

### 2.3.1 Fundamentals and History

Reinforcement Learning is a field of study that emerged from the combination of many other fields like psychology, computer science, and control theory, and it was inspired by the way humans and animals learn from their environment. RL involves an agent that makes decisions, receiving rewards or penalties based on its actions, aiming to maximise positive outcomes over time. This process is deeply rooted in the concept of operant conditioning, a type of learning discovered by psychologists like B. Skinner [20], where animal behaviour is shaped by its consequences.

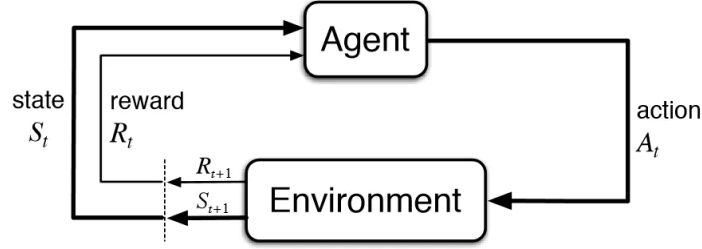
In the 1960s, Markov Decision Processes (MDPs) were introduced, which are frameworks that model decision making where the outcomes are partly random and partly under the control of the decision maker. This new field was the mathematical backbone of RL, as it allowed the development of algorithms that could learn optimal strategies in complex environments. One of the major breakthroughs in RL was the introduction of Q-Learning in the late 1980s, an algorithm capable of learning the value of actions in different states, helping agents make informed decisions without needing a model of the environment. This, along with Temporal Difference Learning, set the stage for the application of RL to a wide range of problems, from playing video games at a superhuman level to optimising financial trading strategies [7].

### 2.3.2 Key Concepts and Agent-Environment Interaction

Reinforcement Learning is about how an agent interacts with its environment. This interaction involves states, actions and rewards. The state  $S_t$  represents the situation of the agent at that time instant. When the agent observes  $S_t$  it chooses an action  $A_t$ . This action causes the environment to transition to a state  $S_{t+1}$ , providing feedback in the form of a reward  $R_{t+1}$ . Refer to Figure 3 for a visual representation of this cyclical interaction [7].

The main objective of the agent is to maximise its sum of rewards, also known as the return. The return is basically the expected gain that can be achieved starting from the state  $S_t$  as expressed by this formula [7]:

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (3)$$



**Figure 3:** Interaction between the agent and the environment in a Reinforcement Learning setting [7].

Here,  $\gamma$  represents the discount factor, indicating the degree to which future rewards are considered important relative to immediate ones. A policy  $\pi$  directs the agent's actions, which can be deterministic, assigning a specific action to each state, or stochastic, assigning a probability distribution over actions. Balancing between exploration, the act of trying new actions to discover their effects, and exploitation, the act of choosing actions that are known to yield high reward, is a fundamental challenge in RL. An important approach to this problem is the  $\epsilon$ -greedy policy, which predominantly exploits the best-known action but occasionally explores random actions, thus ensuring both robustness and continual learning [7].

The fundamental concept behind Reinforcement Learning lies in the Markov Decision Process (MDP) framework, defined by the tuple  $(S, A, P, R, \gamma)$ . Here,  $S$  represents all possible states in the environment,  $A$  denotes the actions the agent can take, and  $P$  is the transition probability, indicating the likelihood of moving from state  $s_t$  to state  $s_{t+1}$  after taking action  $a$ .  $R$  is the return received for this transition, and  $\gamma$  is the discount factor. MDPs utilise the Markov property, ensuring that the next state depends only on the current state and action, without need to consider any past states or actions, as shown in this formula [7]:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1}|s_t, a_t) \quad (4)$$

The reward function is very important as it captures the agents objectives. It offers a signal that directs the agent towards achieving the goals of a task, but it does not prescribe how to accomplish them. For example, in the trading context, rewards can mirror gains and losses, guiding the agent to adopt strategies that maximise profits [7].

### 2.3.3 Value Functions and Q-Functions

The value function for a policy  $\pi$ , denoted as  $V^\pi(s)$ , represents the expected return when starting in state  $s$  and following policy  $\pi$  afterwards. It is formulated as follows [7]:

$$V^\pi(s) = \mathbb{E} \left[ \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, \pi \right] \quad (5)$$

Here,  $\gamma$  is the discount factor, which weighs the importance of future rewards compared to immediate rewards, and  $\mathbb{E}$  denotes the expected value, reflecting the average of all possible outcomes weighted by their probabilities. The Q-function for a policy  $\pi$ , denoted as  $Q^\pi(s, a)$ , represents the expected return for taking an action  $a$  in state  $s$  and following policy  $\pi$  afterwards. It is formulated as follows [7]:

$$Q^\pi(s, a) = \mathbb{E} \left[ \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a, \pi \right] \quad (6)$$

The Bellman equations, named after Richard Bellman, who first formulated them, provide a recursive decomposition for the total cumulative reward an agent can expect to receive starting from a particular state or state-action pair. These equations are fundamental to dynamic programming techniques in RL, as they express the relationship between the value of the current state or state-action pairs as the expected

values of subsequent states or state-action pairs. The recursive nature of the Bellman equations allows for the iterative improvement of the estimated values, essential for the convergence of RL algorithms. This is expressed as follows [7]:

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')] \quad (7)$$

$$Q^\pi(s, a) = \sum_{s' \in S} P(s'|s, a) \left[ R(s, a, s') + \gamma \sum_{a' \in A} \pi(a'|s') Q^\pi(s', a') \right] \quad (8)$$

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) Q^\pi(s, a) \quad (9)$$

The relationship pointed in Equation 9 indicates that the value of a state under policy  $\pi$  is the expected value of the Q-function over all possible actions, weighted by the policy's action probabilities. For optimal policies, we define the optimal value function  $V^*(s)$  and the optimal Q-function  $Q^*(s, a)$ , which represent the maximum possible returns achievable from any state or state-action pair, respectively. Following the Bellman equation, it is possible to obtain [7]:

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')] \quad (10)$$

$$Q^*(s, a) = \sum_{s' \in S} P(s'|s, a) \left[ R(s, a, s') + \gamma \max_{a' \in A} Q^*(s', a') \right] \quad (11)$$

$$V^*(s) = \max_{a \in A} Q^*(s, a) \quad (12)$$

Through the iterative application of these equations, RL algorithms gradually approximate the value and Q-functions, guiding the agent towards the optimal policy that maximises the expected cumulative reward [7].

#### 2.3.4 Methodological Variants and Learning Methods

The field of reinforcement learning can be divided into categories based on the properties of the learning method: model-based versus model-free, on-policy versus off-policy strategies and value-based versus policy-based techniques.

In model-based RL, algorithms create a model of the environment to simulate and predict the outcomes of actions. This allows the agent to plan by forecasting the states. This is possible when the environment is simple enough to be modelled. On the other hand, in model-free RL, algorithms directly learn a policy or value function from their interactions with the environment without explicitly modelling it. Model-free approaches are often chosen for environments where modelling is challenging or when an accurate model is not available. Within the realm of model-free RL, there are on-policy strategies, which learn about the policy that is currently used to make decisions, and off-policy strategies, which explore a different policy while aiming to learn an optimal one [7].

Another important distinction lies between value-based (also designated as critic-only) and policy-based (also designated as actor-only) learning approaches. On the one hand, value-based methods, like Temporal Difference Learning (TD), Q-Learning and SARSA Learning, focus on estimating the value of states or state-action pairs. These algorithms derive policies implicitly from these values. SARSA Learning uses an on-policy strategy, while Q-Learning and TD Learning use an off-policy strategy. On the other hand, policy-based methods, such as REINFORCE Learning, directly optimise the policy itself, which is particularly useful when dealing with continuous action spaces. In particular, Actor-Critic learning combines elements of both policy-based and value-based approaches. It has a policy function (the actor) and a value function (the critic), which provides the stability and efficiency of value-based methods along with direct policy improvement. This collaboration between the actor and the

---

**Algorithm 2** Actor-Critic

---

**Require:**  $\pi(s, a; \theta)$ : a differentiable policy,  $Q(s, a; w)$ : a differentiable  $Q$ -function,  $N$ : number of iterations and learning rates  $\beta^\theta > 0$  and  $\beta^w > 0$   
Initialize policy parameter  $\theta^{(0)}$  and  $Q$ -function parameter  $w^{(0)}$   
Initialize state  $s$   
**for**  $n = 0, 1, \dots, N - 1$  **do**  
    Sample  $a \sim \pi(s, a; \theta^{(n)})$   
    Take action  $a$ , observe state  $s'$  and reward  $r$   
    Sample action  $a' \sim \pi(s', a; \theta^{(n)})$   
    Compute the TD error:  $\delta \leftarrow r + \gamma Q(s', a'; w^{(n)}) - Q(s, a; w^{(n)})$   
    Update  $w$ :  $w^{(n+1)} \leftarrow w^{(n)} + \beta^w \delta \nabla_w Q(s, a; w^{(n)})$   
    Update  $\theta$ :  $\theta^{(n+1)} \leftarrow \theta^{(n)} + \beta^\theta \delta \nabla_\theta \ln \pi(s, a; \theta^{(n)})$   
     $s \leftarrow s', a \leftarrow a'$   
**end for**

---

critic results in improved policy updates, especially beneficial for complex environments [7]. Refer to Algorithm 2 for a detailed implementation.

Although the Puterman theorem guarantees a deterministic stationary optimal policy for Markov Decision Processes (MDPs), such assurances do not extend to complex, non-stationary environments like financial markets [7]. Forex markets, in particular, are influenced by historical data in ways that standard state representations in MDPs do not account for. Consequently, RL in these markets relies on approximated models, utilising technical indicators and feature engineering to integrate essential historical information into the state representation in order to try to approximate the Markov property. This is why algorithms like the Actor-Critic learning are of special importance, because they act as a connection between reinforcement learning and neural networks, as we will see in the next section.

## 2.4 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) merges reinforcement learning's decision-making with the pattern recognition capabilities of neural networks, making it especially powerful for handling complex, high-dimensional data in environments like the Forex market. DRL empowers trading algorithms with adaptability, positioning them to potentially revolutionise trading strategies.

### 2.4.1 Fundamentals and History

The rise of DRL as a new field within machine learning gained momentum when it became clear that neural networks could greatly enhance the capabilities of reinforcement learning agents in dealing with and understanding complex environments. The groundbreaking work of DeepMind [4], which demonstrated the success of the Double Deep Q-Network (DDQN) algorithm in mastering a range of Atari 2600 games using inputs, marked a significant breakthrough for DRL. This achievement not only validated the efficacy of DRL in mastering high-dimensional state spaces but also highlighted its potential applicability to financial markets. In this context, the challenge lies not only in interpreting large amounts of data but also in extracting strategic insights that can guide real-time trading decisions. This historical advance, along with ongoing research, has sparked interest in applying DRL to Forex trading, known to be a very volatile and dynamic environment.

### 2.4.2 Deep Value-Based RL Algorithms: Deep Q-Networks

Deep learning has transformed traditional value-based reinforcement learning methods, giving rise to what is known today as deep value-based methods. These advanced algorithms employ neural networks, known for their function approximation capabilities, to manage and interpret larger state spaces. Such an approach enables agents to learn and operate effectively in environments with complex, high-dimensional

sensory input, a task that conventional value-based methods could not scale too efficiently. Neural Fitted Q-Learning was one of the early techniques that laid the foundation for deep value-based learning [2]. It utilises neural networks to approximate the Q-function, which represents the expected return of a given state-action pair. By fitting a neural network to the Q-values obtained from completed episodes, it significantly enhanced the agent's ability to generalise across states in large and continuous spaces. However, the field of deep value-based methods truly came into prominence with the introduction of the Deep Q-Network (DQN) algorithm [3]. DQN integrated key innovations such as a separate target network for stabilising the learning targets and an experience replay mechanism to break the temporal correlations within training batches. This approach allowed DQN to effectively learn control policies directly from high-dimensional inputs, a significant jump forward in the application of RL. Following DQN, improvements have been made such as the Double Deep Q-Network (DDQN) to address limitations in the DQN architecture [4]. DDQN mitigates the overestimation bias of the action values by decoupling the action selection from the action evaluation. This is achieved by employing two networks: a primary network for action selection and a target network for evaluation, enhancing the stability and accuracy of the value estimation process. Although deep value-based methods are foundational and revolutionary, they will not be the focus of this research.

### 2.4.3 Deep Policy-Based RL Algorithms: Deep Policy Gradients

Deep policy-based RL algorithms excel in environments with continuous action spaces. Unlike value-based methods that assign a value to each potential action, policy-based approaches directly optimise the policy to maximise expected returns. This quality is invaluable in the domain of Forex trading, characterised by a vast and continuous action space that demands both precision and adaptability. Among these approaches, the Deep Deterministic Policy Gradient (DDPG) algorithm stands out as a model-free off-policy actor-critic algorithm specifically designed for continuous action spaces [5]. By combining the direct policy optimisation of policy-based methods with the evaluation power of value-based approaches, DDPG achieves a stable and robust learning process. The algorithm consists of an actor component for selecting actions and a critic component that evaluates the actions by estimating their Q-values. In Algorithm 3 we can observe how DDPG uses neural networks to approximate both the actor and critic functions. This enables handling complex input dimensions commonly encountered in Forex markets. The utilisation of a replay buffer for experience storage and the strategic sampling of mini-batches are key features that enhance learning stability by mitigating the correlation between sequential data points. Furthermore, DDPG incorporates target networks, which are gradually updated versions of the actor and critic, to solidify the learning targets and encourages gradual improvement.

Although we cannot guarantee the convergence of the DDPG due to the partially observable nature of the Forex markets, its design for continuous policy adaptation presents a good approach to developing sophisticated trading strategies. The adaptability of DDPG to the evolving conditions of financial markets and its ability to identify complex patterns highlight its potential in algorithmic trading.

## 3 Related Work

This section reviews the existing literature on the application of deep reinforcement learning (DRL) in financial trading, covering a variety of markets, datasets, neural network architectures, reinforcement learning paradigms, optimisation and training techniques and key findings. The research process started with surveys to acquire a quick understanding of the current state-of-the-art and to properly redirect the attention towards the right papers. Subsequently, the article research was carried out and all relevant information was summarised and put in table format.

### 3.1 Survey Research: Identifying the State-of-the-Art

One of the most important papers read is [21] establishing a really good foundation for the investigation. In this survey, all the important background is presented, from defining the theoretical ground of



---

**Algorithm 3** Deep Deterministic Policy Gradient

---

**Require:** an actor  $\pi(s; \theta)$ , a critic network  $Q(s, a; \phi)$ , learning rates  $\beta$  and  $\hat{\beta}$ , initial parameters  $\theta^{(0)}$  and  $\phi^{(0)}$   
Initialize target network parameters  $\hat{\phi} \leftarrow \phi^{(0)}$  and  $\hat{\theta} \leftarrow \theta^{(0)}$ , replay buffer  $\mathcal{B}$   
**for** episode = 1,  $M$  **do**  
    Initialize a random process  $\mathcal{N}$  for action exploration  
    Receive initial observation state  $s_1$   
    **for**  $t = 1, T$  **do**  
        Select action  $a_t = \pi(s_t; \theta) + \mathcal{N}_t$  according to the current policy and exploration noise  
        Execute action  $a_t$  and observe reward  $r_t$  and new state  $s_{t+1}$   
        Store transition  $(s_t, a_t, r_t, s_{t+1})$  in  $\mathcal{B}$   
        Sample a random minibatch of  $N$  transitions  $(s_i, a_i, r_i, s_{i+1})$  from  $\mathcal{B}$   
        Set  $y_i = r_i + \gamma Q'(s_{i+1}, \pi'(s_{i+1}; \hat{\theta}); \hat{\phi})$   
        Update critic by minimizing the loss:  $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i; \phi))^2$   
        Update the actor policy using the sampled policy gradient:  
         $\nabla_{\theta} J \approx \frac{1}{N} \sum_i \nabla_a Q(s_i, a; \phi)|_{s=s_i, a=\pi(s_i)} \nabla_{\theta} \pi(s; \theta)|_{s_i}$   
        Update the target networks:  
         $\hat{\phi} \leftarrow \tau \phi + (1 - \tau) \hat{\phi}$   
         $\hat{\theta} \leftarrow \tau \theta + (1 - \tau) \hat{\theta}$   
    **end for**  
**end for**

---

reinforcement learning and neural networks to defining how deep reinforcement emerged from combining these two fields of study and how it is being applied to various financial applications. In it, all the relevant formulas and mathematical foundation is introduced with pseudocodes from the most recent advances in critic-only, actor-only and actor-critic algorithms of DRL, along with the respective references for the papers that first implemented them.

Another very important survey is [22] which, contrary to the previous one, is less about theoretical foundations on the matter and more about highlighting current practical applications of these DRL algorithms in financial applications. In it, various articles with practical implementations were studied and summarised, allowing the reader to understand how the research space is being explored and identify possible research gaps. Most importantly, we were allowed to understand what to look for when conducting our own research in the current research space, namely what datasets are being used, what neural network algorithms are beneath the reinforcement learning architecture, what state spaces, action spaces and reward functions were used, what training mechanisms were used to cope with bias-variance and exploration-exploitation trade-offs and what optimisation techniques were implemented to ensure a better convergence of the models.

Other surveys like [23] and [24] were investigated, allowing us to solidify our understanding of the current state-of-the-art in terms of practical implementations of trading systems using DRL and what to look for when conducting our research.

## 3.2 Article Research: Summarising Different Implementations

After carrying out the research in broader terms with the surveys, a more specific investigation was conducted looking for articles with practical implementations. All relevant features of the implementations were summarised in Table 9 and Table 10, both tables being relative to the same articles but divided into basic and advanced features.

### 3.2.1 Forex Market Implementations

Several studies have focused on applying DRL techniques to the Forex market, using various neural network architectures and reinforcement learning frameworks. [25] implemented a Deep Q-Network

(DQN) algorithm with a Fully Connected Neural Network (FNN) beneath it on raw tick data over a period from 2010 to 2017. The model was limited to trade only after a predetermined number of ticks, resulting in a timeframe of around two hours. This study demonstrated significant outperformance over traditional benchmarks, highlighting the efficacy of the DQN approach in high-frequency scenarios. Another article is [26] exploring the application of a DQN model with a Recurrent Neural Network (RNN), more specifically a Long-Short Term Memory Network (LSTM). This algorithm is generically called a Deep Recurrent Q-Network (DRQN) and the author applied on 15-minute OHLCV data from 2012 to 2017. Their research resulted in substantial outperformance of the model compared to benchmarks, even when explicit and implicit trading costs were considered, emphasising the importance of RNN in the handling of sequential data, which is crucial to capture temporal dependencies in financial time series. Similarly, [27] implemented another DRQN model in the minute timeframe studied between 2004 and 2018. The study concluded that the model outperformed conventional benchmarks but had abnormally high accuracy, implying some sort of look-ahead bias. It also showed a high maximum drawdown, which implies that the DQN model, even with RNN (LSTM), might not be the appropriate solution for very low timeframes.

### 3.2.2 Stock Market Implementations

A substantial body of research has focused on applying DRL to stock trading, exploring various algorithms and neural network architectures. [28] utilised the Deep Deterministic Policy Gradient (DDPG) algorithm with a RNN (LSTM) on daily OHLCV and news data for approximately five months, the exact time period is not explicitly outlined. The inclusion of news data as a feature was shown to improve model performance, demonstrating the value of incorporating fundamental information into trading strategies. [29] employed the DDPG algorithm with FNN on monthly stock data from 2005 to 2019. This study demonstrated the ability of the model to outperform benchmarks even when accounting for explicit and implicit costs. The integration of cost considerations into the model evaluation highlights the practical applicability of the DDPG algorithm in stock trading. This study also emphasises the adaptability of this type of models to higher timeframes. [30] explored an ensemble approach in daily OHLCV data from 2016 to 2020 that combined three actor-critic DRL algorithms, namely Deep Deterministic Policy Gradient (DDPG), Advantage Actor Critic (A2C) and Proximal Policy Optimization (PPO). Despite the innovative ensemble methodology, the study reported unstable returns and underperformance relative to benchmarks, suggesting that while ensemble methods may diversify strategies, they can also introduce instability. This highlights the need for further research to improve the stability of ensemble DRL models in stock trading. [31] analysed two distinct architectures, DQN with FNN and Asynchronous Advantage Actor Critic (A3C) with LSTM, in minute-level OHLCV data for stocks and futures from 2010 to 2020. Both models significantly outperformed the benchmarks, and A3C showed superior results. This research underscores the efficacy of the A3C model in high-frequency trading environments, demonstrating its robustness and adaptability in capturing market dynamics, advantages of the parallel and asynchronous training the model provides. [32] explored algorithmic trading strategies across both stocks and cryptocurrencies, utilizing OHLCV data over a daily timeframe from 2010 to 2021. The study employed a FNN with an Actor-Critic approach, implementing another innovative algorithm designated Twin-Delayed Deep Deterministic Policy Gradient (TD3). The findings indicated superior performance in cryptocurrencies, though the approach underperformed in stock trading. [33] applied DRL to stock trading using OHLCV data with a daily frequency from 2000 to 2020. They adopted a Recurrent Neural Network (RNN) with LSTM cells and an Actor-Critic strategy, leveraging DDPG and PPO models. The study revealed that both models significantly outperformed the benchmark, with PPO exhibiting better stability and performance. [34] presented a generalized DRL framework for stock trading with a focus on out-of-sample generalization. Using daily OHLCV data from 2006 to 2022, they implemented a Critic-Only approach with a DQN model, achieving superior performance compared to the benchmark, particularly in unseen assets. [35] investigated the application of DRL in stock trading under different market conditions. Utilizing daily OHLCV data from 2012 to 2019, they implemented a Critic-Only framework with a DQN model. Their results demonstrated consistent outperformance of the benchmark in both stable and volatile market conditions. [36] focused on adaptive DRL strategies for stock trading,

employing daily OHLCV data from 2008 to 2018. They utilized Gated Recurrent Units (GRU), a type of RNN, with both Critic-Only and Actor-Critic approaches, deploying GDQN and GDPG models. The study showed that both models outperformed the benchmark, with GDPG achieving superior results, once again highlighting the importance of time-dependent features in trading systems. [37] implemented a practical DRL approach for stock trading using daily OHLCV data from 2009 to 2018. They employed an FNN with an Actor-Critic strategy and DDPG model, demonstrating significant outperformance compared to the benchmark. [38] explored ensemble DRL methods for stock trading, leveraging daily OHLCV data from 2009 to 2020. They combined DDPG, A2C, and PPO models within an Actor-Critic framework, finding that the ensemble significantly outperformed the benchmark, with PPO showing the best results, followed by A2C and DDPG. [39] examined deep reinforcement learning strategies for stock trading, using daily OHLCV data from 2009 to 2021. They employed an FNN with an Actor-Critic approach, implementing both DDPG and A2C models. Their findings indicated that both models outperformed the benchmark, with DDPG exhibiting superior convergence and stability due to experience replay and noise handling.

### 3.2.3 Other Market Implementations

The application of Deep Reinforcement Learning (DRL) in other financial markets has also been an area of active research, demonstrating the versatility of these models across different asset classes. [40] investigated the use of DRL for the trading of ETFs, using daily OHLCV data from 2000 to 2018. They used the DQN algorithm with both FNN and RNN (with LSTM cells, using a Critic-Only strategy with DQN and DRQN models. The study found that both models outperformed the benchmark, and DRQN showed superior results. [41] explored LSTM-DDPG models for ETF trading, leveraging daily OHLCV data from 2002 to 2020. They utilised a RNN (LSTM) with an Actor-Critic approach and a DDPG model. This DRL model allows continuous action spaces, which allowed the author to test two variants of the model, one with fixed position sizing and one with dynamic position sizing. The study concluded that both approaches outperformed the benchmark even with the explicit and implicit cost considered, but the variable position sizing variant had superior results. This emphasises the importance of risk management in trading systems and reinforces the idea that time-dependent neural networks are of special importance in time series analysis. [42] applied DRL to futures trading, using daily OHLCV data from 2005 to 2019. They used an RNN (LSTM) with an Actor-Critic framework and A2C model, achieving significant outperformance compared to the benchmark, even after accounting for explicit and implicit costs.

### 3.2.4 Key Findings

Looking at the basic and advanced features in Table 9 and Table 10, several key patterns and insights have emerged regarding different studies. Regarding the state space, a significant observation is that most papers that achieve notable results incorporate technical analysis as part of their state space. This approach helps to capture market trends and momentum, enhancing the model's ability to make informed trading decisions. Articles like [25], [26], and [38] highlighted the importance of integrating technical analysis with raw market data to improve performance. Regarding the action space, the discrete option (1, 0, -1) was a common choice, especially in scenarios without variable position sizing. This scheme simplifies the decision-making process for buying, holding, or selling, making it intuitive and effective. Studies such as [27], [28], and [37] used this discrete action space convention with positive results. Regarding the reward function, the sharpe ratio emerged as a preferred option due to its emphasis on risk-adjusted returns. Despite its complexity, it provides a robust measure of performance by balancing returns against volatility. Articles including [30], [33], and [41] demonstrated the advantages of using the sharpe ratio over simpler reward metrics such as total return. Regarding bias-variance techniques, common methods included feature engineering, normalisation, regularisation, and tweaking of the neural network architecture and parameters during training. Examples include [31] which utilized denoising and [35] which implemented normalization and regularization techniques. Regarding exploration-exploitation techniques, many studies employed  $\epsilon$ -greedy policies and added Gaussian noise

to encourage exploration while balancing exploitation of known strategies. Studies such as [36] and [32] successfully incorporated these techniques. Regarding training techniques, a robust training regimen often included a train-validation-test split, with some articles adopting cross-validation for enhanced robustness. One notable method was online learning, which simulates live trading environments and allows continuous learning. [26] showcased online learning, providing information on its effectiveness in dynamic market conditions.

## 4 Proposed Solution

In this section, we present the identified research gaps, an overview of the proposed solution, and the innovative aspects of this solution. The proposed solution aims to contribute to the application of Deep Reinforcement Learning (DRL) in Forex trading by addressing specific research gaps and introducing novel approaches to model architecture, training, and optimisation techniques.

### 4.1 Research Gaps

Several research gaps have been identified. Firstly, the application of the Deep Deterministic Policy Gradient (DDPG) in Forex markets is underexplored, despite its advantages in handling continuous action spaces. Secondly, while Recurrent Neural Networks (RNN), especially Long-Short Term Memory (LSTM) networks, are effective for time-dependent data, their integration with DDPG in Forex trading remains limited. Thirdly, enhancing the reward function from total return to the Sharpe ratio for better risk-adjusted returns is needed. Furthermore, online learning for real-time market adaptation is relatively unexplored, and advanced risk and portfolio management techniques, such as variable position sizing within the DDPG framework, require further research.

### 4.2 Proposed Solution

The proposed solution addresses these gaps by developing a Forex trading system that integrates DDPG with an LSTM architecture, incorporates the Sharpe ratio as the reward function, and supports online learning for continuous market adaptation. This approach aims to leverage the strengths of advanced DRL techniques to enhance trading performance and robustness.

#### 4.2.1 Model Innovation

The application of DDPG to the Forex market represents a significant innovation in this field. DDPG is at the forefront of advancements in DRL algorithms, known for its ability to handle high-dimensional continuous action spaces effectively, making it particularly suitable for complex trading environments. Initially, the model will be tested with fixed position sizing. However, given DDPG’s capability to support continuous action spaces, we will explore the potential of variable position sizing to enhance trading strategies. The initial implementation will utilize a vanilla FNN underneath the DDPG model. If time permits, we plan to integrate an RNN (LSTM) architecture due to its superiority in capturing sequential dependencies and temporal patterns, which are critical in financial time series analysis. Moreover, we will begin with the total return as the reward function but aim to test the Sharpe ratio if feasible, due to its focus on risk-adjusted performance, offering a more comprehensive evaluation of trading strategies. This plan includes testing eight different models: DDPG with FNN and RNN (LSTM), each with fixed and variable position sizing, using both total return and Sharpe ratio as reward functions.

#### 4.2.2 Training Innovation

A key innovative aspect of the proposed solution is the direct integration with the cTrader platform [1], a prominent trading platform supported by many well-known brokers and a major competitor to MetaTrader4/5 [43]. cTrader’s cross-broker compatibility allows researchers to select brokers that best align with their requirements, such as commission rates, swaps, and spreads. This project proposes

| Reference | Market          | Dataset     | Timeframe  | Period      | NN                     | RL                        | DRL                 | Key Findings                                                                                                                                |
|-----------|-----------------|-------------|------------|-------------|------------------------|---------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| [25]      | Forex           | Raw Ticks   | ≈ 2 Hours  | 2010 - 2017 | FNN                    | Critic-Only               | DQN                 | Significantly outperformed the benchmark.                                                                                                   |
| [26]      | Forex           | OHLCV       | 15 Minutes | 2012 - 2017 | RNN (LSTM)             | Critic-Only               | DRQN                | Significantly outperformed the benchmark even with explicit and implicit costs.                                                             |
| [27]      | Forex           | OHLCV       | Minute     | 2004 - 2018 | RNN (LSTM)             | Critic-Only               | DRQN                | Outperformed the benchmark, but had abnormally high accuracy and maximum drawdown.                                                          |
| [28]      | Stocks          | OHLCV, News | Daily      | ≈ 5 Months  | RNN (LSTM)             | Actor-Critic              | DDPG                | Outperformed the benchmark and News are important features.                                                                                 |
| [29]      | Stocks          | OHLCV       | Monthly    | 2005 - 2019 | FNN                    | Actor-Critic              | DDPG                | Outperformed the benchmark even with explicit and implicit costs.                                                                           |
| [30]      | Stocks          | OHLCV       | Daily      | 2016 - 2020 | FNN                    | Actor-Critic              | DDPG, A2C, PPO      | The ensemble underperformed the benchmark and had unstable returns.                                                                         |
| [31]      | Stocks, Futures | OHLCV       | Minute     | 2010 - 2020 | FNN (SDAE), RNN (LSTM) | Critic-Only, Actor-Critic | DQN, A3C            | Both significantly outperformed the benchmark and A3C had superior results and convergence.                                                 |
| [32]      | Stocks, Cryptos | OHLCV       | Daily      | 2010 - 2021 | FNN                    | Actor-Critic              | DDPG (TD3)          | Outperformed the Cryptos benchmark but underperformed the Stocks benchmark.                                                                 |
| [33]      | Stocks          | OHLCV       | Daily      | 1992 - 2020 | FNN                    | Critic-Only, Actor-Critic | DQN, DDPG, PPO, A2C | All outperformed the benchmark, PPO and DQN had superior results and more consistent, DDPG and A2C performance was below expectation.       |
| [34]      | Stocks          | OHLCV       | Daily      | 2006 - 2022 | FNN                    | Critic-Only               | DQN                 | Outperformed the benchmark with good generalization power in out-of-sample assets.                                                          |
| [35]      | Stocks          | OHLCV       | Daily      | 2012 - 2019 | FNN                    | Critic-Only               | DQN                 | Outperformed the benchmark with consistent results in stable and volatile markets.                                                          |
| [36]      | Stocks          | OHLCV       | Daily      | 2008 - 2018 | RNN (GRU)              | Critic-Only, Actor-Critic | GDQN, GDPG          | Both outperformed the benchmark with consistent results in stable and volatile markets. GDPG had superior results.                          |
| [37]      | Stocks          | OHLCV       | Daily      | 2009 - 2018 | FNN                    | Actor-Critic              | DDPG                | Outperformed the benchmark.                                                                                                                 |
| [38]      | Stocks          | OHLCV       | Daily      | 2009 - 2020 | FNN                    | Actor-Critic              | DDPG, A2C, PPO      | The ensemble significantly outperformed the benchmark with explicit costs, then PPO had the best results, followed by A2C and finally DDPG. |
| [39]      | Stocks          | OHLCV       | Daily      | 2009 - 2021 | FNN                    | Actor-Critic              | DDPG, A2C           | Both outperformed the benchmark. DDPG had superior results, convergence and stability due to experience replay and noise.                   |
| [40]      | ETFs            | OHLCV       | Daily      | 2000 - 2018 | FNN, RNN (LSTM)        | Critic-Only               | DQN, DRQN           | Both outperformed the benchmark and DRQN had superior results.                                                                              |
| [41]      | ETFs            | OHLCV       | Daily      | 2002 - 2020 | RNN (LSTM)             | Actor-Critic              | DDPG                | Outperformed the benchmark using variable position sizing and explicit and implicit costs.                                                  |
| [42]      | Futures         | OHLCV       | Daily      | 2005 - 2019 | RNN (LSTM)             | Actor-Critic              | A2C                 | Outperformed the benchmark with explicit and implicit costs.                                                                                |

**Table 9:** Summary of the basic features of related work articles.

| Reference | State Space                             | Action Space                  | Reward Function                 | Bias/Variance                           | Exploration/Exploitation                    | Training Techniques                               |
|-----------|-----------------------------------------|-------------------------------|---------------------------------|-----------------------------------------|---------------------------------------------|---------------------------------------------------|
| [25]      | Ask, Bid, Volume, Technical Analysis    | Discrete (1, 0, -1)           | Total Return                    | Tweak Network, Early Stopping           | Policy $\epsilon$ -greedy                   | Cross-Validation                                  |
| [26]      | OHLCV, Technical Analysis               | Discrete (1, 0, -1)           | Total Return                    | Tweak Replay Memory, Tweak RNN Sequence | Policy $\epsilon$ -greedy                   | Online Learning                                   |
| [27]      | OHLCV                                   | Discrete (1, 0, -1)           | Total Return                    | Tweak Network                           | -                                           | Train-Validate-Test Split                         |
| [28]      | OHLCV, Sentiment and Technical Analysis | Discrete (1, 0, -1)           | Total Return                    | Tweak Network                           | Policy $\epsilon$ -greedy, Shuffling        | Cross-Validation                                  |
| [29]      | Log Returns, Volume                     | Discrete (1, 0, -1)           | Total Return                    | -                                       | -                                           | Pre-Train with GA                                 |
| [30]      | OHLCV, Technical Analysis               | Discrete (1, 0, -1)           | Sharpe Ratio                    | Ensemble Selection                      | Ensemble Selection                          | Cross-Validation                                  |
| [31]      | OHLCV, Metrics, Technical Analysis      | Discrete (-n, ..., 0, ..., n) | Total Return                    | Denosing                                | Gaussian Noise                              | Adaptive Learning Rate                            |
| [32]      | OHLCV                                   | Continuous [-1, 1]            | Log-Return                      | Normalisation, Regularisation           | Policy $\epsilon$ -greedy Gaussian Noise    | Train-Validate-Test Split                         |
| [33]      | OHLCV                                   | Discrete (1, 0, -1)           | Sharpe Ratio                    | Wavelet Transform                       | -                                           | Train-Validate-Test Split                         |
| [34]      | OHLCV, Macroeconomic Metrics            | Discrete (1, 0, -1)           | Total Return, (1, 0, -1)        | Normalisation, Regularisation           | Policy $\epsilon$ -greedy (Adaptive)        | Adaptive Synchronization                          |
| [35]      | OHLCV                                   | Discrete (1, -1)              | Total Return                    | Normalisation, Regularisation           | Duplicate Environment (Inverse Positioning) | Adaptive Learning Rate, Huber Loss, Gradient Clip |
| [36]      | OHLCV, Technical Analysis               | Discrete (1, 0, -1)           | Total Return                    | Normalisation, Regularisation           | Policy $\epsilon$ -greedy Gaussian Noise    | Train-Test Split                                  |
| [37]      | OHLCV                                   | Discrete (1, 0, -1)           | Total Return                    | Tweak Network                           | Gaussian Noise                              | Train-Validate-Test Split                         |
| [38]      | OHLCV, Technical Analysis               | Discrete (-n, ..., 0, ..., n) | Total Return (Turbulence Index) | Ensemble Selection                      | Ensemble Selection                          | Train-Validate-Test Split                         |
| [39]      | OHLCV, Technical Analysis               | Discrete (1, 0, -1)           | Total Return                    | Normalisation                           | Gaussian Noise                              | Train-Test Split                                  |
| [40]      | OHLCV                                   | Discrete (1, 0, -1)           | Total Return                    | Tweak Learning Rate                     | -                                           | -                                                 |
| [41]      | OHLCV                                   | Continuous (Position Sizing)  | Sharpe Ratio                    | -                                       | Gaussian Noise                              | Train-Validate-Test Split                         |
| [42]      | OHLCV, Technical Analysis               | Continuous [-1, 1]            | Total Return (with Volatility)  | Normalisation                           | -                                           | Cross-Validation                                  |

**Table 10:** Summary of the advanced features of related work articles.

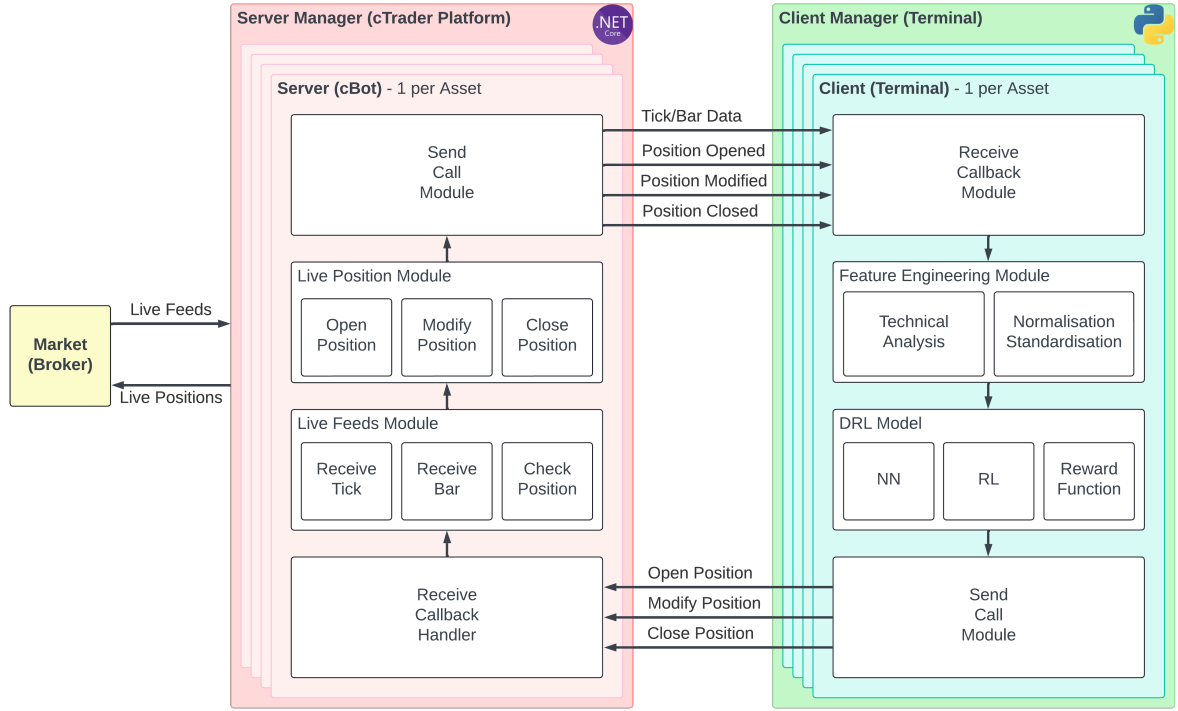
building a server in .NET and a client in Python, where the server handles data feeds and trading operations (e.g., opening, modifying, and closing positions), and the client processes these feeds and sends back trading decisions. This setup enables online training, a superior technique that closely simulates live trading conditions, allowing the model to adapt to real-time market changes. In addition, it facilitates visual backtesting of strategies and seamless deployment in live trading contexts. This approach aims to combine the robustness of cTrader’s infrastructure with the flexibility and analytical capabilities of Python, allowing accurate performance results that account for real explicit and implicit costs.

#### 4.2.3 Optimization Innovation

The proposed solution introduces several innovative optimisation techniques to improve the convergence and generalisation power of the model. We will start by testing the model on an hourly timeframe for the 7 major Forex pairs, an underexplored area, and, if feasible, extend the testing to a daily timeframe. The state space will be enriched with technical analysis components in addition to OHLCV data to provide a comprehensive view of market conditions and achieve the Markov property, which is essential for effective reinforcement learning. All features will undergo rigorous engineering; we plan to apply normalisation techniques to standardise the data, making it easier for the neural networks to process and learn effectively. Regularisation techniques and adaptive learning rates will be used in neural network models to mitigate bias-variance issues, enhancing the generalisation capabilities of the model. In the reinforcement learning models, we will utilise  $\epsilon$ -greedy policies and Gaussian noise to balance exploration and exploitation, encouraging the discovery of optimal trading strategies while leveraging known profitable actions. These optimisation strategies are designed to enhance the model’s performance, stability, and adaptability, ensuring it can effectively navigate the complexities of the Forex market.

#### 4.2.4 System Architecture

The system architecture diagram for the proposed solution is shown in Figure 4. This architecture consists of three major components: the Market, the Server Manager and the Client Manager. In this framework, we are implementing a one-to-one Client-Server architecture, meaning that for each asset there is one Client that generates signals and one Server that handles the data feeds and positioning operations. Therefore, this is a one-to-one communication and there exist as many Servers as Clients. Then, each set of Servers and Clients is managed by a Server Manager and a Client Manager, respectively, capable of starting, monitoring and shutting all of them. Going in more depth on the details, firstly, the Market component represents a broker with access to the cTrader platform. There are currently several prestigious and regulated alternatives, but for this research we will use IC Markets EU as our broker [44]. Secondly, the Server Manager component represents a cTrader platform running on the local machine. Within cTrader platform there are several Servers running, where each one of them basically consists on a cBot implementation, a .NET program that allows algorithmic trading within cTrader. Each Server includes several submodules, namely: the Call module, which sends requests to the Client; the Live Position module, responsible for opening, modifying, and closing positions; and the Live Feeds module, which receives Tick and Bar data and checks how new data affected currently opened positions; and the Callback Handler module, which handles requests from the Client. Thirdly, the Client Manager component, implemented in Python, consists of a program that manages individual Clients, each one connected to a single server and with similar submodules to it, namely: the Callback module that receives data from the Server including Tick/Bar data and position information; the Feature Engineering module that cleans, normalises, standardises and adds technical analysis to the data; the DRL Model module, including a Neural Network submodule (either FNN or LSTM), a RL submodule (actor-critic), and a Reward Function submodule (Total Return or Sharpe Ratio), which receives the processed data and generates actions (long, short, neutral); and the Call module that sends these actions to the Server. Upon successful execution of actions by the Server, the Client waits for the reward, which is generated when positions are closed. This architecture ensures robust, real-time adaptation and performance evaluation of the DRL-based trading system in an online training paradigm.



**Figure 4:** Proposed DRL-based Forex Trading System Architecture

## 5 Experimental Results

This section outlines the methodology for evaluating the performance of the proposed trading models. Starting by defining a meaningful benchmark, we will use a combination of baseline strategies. A "Buy and Hold" strategy is less applicable to the Forex market, so we will employ a Random Signal Strategy and a Rule-Based Strategy. The Random Signal Strategy generates random buy, sell, or hold signals, providing a naive benchmark. The Rule-Based Strategy will use simple moving averages that offer a structured and realistic comparison. Performance will be evaluated using the metrics in Table 11 from the cTrader platform :

| Metric                                    | Description                                  |
|-------------------------------------------|----------------------------------------------|
| Net Profit                                | Total profit or loss generated               |
| Profit Factor                             | Ratio of gross profit to gross loss          |
| Commission and Swap                       | Costs affecting net profitability            |
| Max Balance and Equity Drawdown           | Maximum observed losses                      |
| Total Trades and Winning Trades           | Indicators of activity and success           |
| Max Consecutive Winning and Losing Trades | Consistency Indicators                       |
| Largest Winning and Losing Trades         | Potential for significant gains or drawdowns |
| Average Trade                             | Measure of trading efficiency                |
| Sharpe Ratio and Sortino Ratio            | Measures of Risk-Adjusted Performance        |

**Table 11:** Summary of the performance metrics that will be used.

The process includes using accurate historical Forex data to assess performance, providing insights



into alpha generation and risk management. We will compare the results of the DDPG model with baseline strategies using the selected metrics to highlight the relative performance of the model. In addition, we will use the tools of cTrader to analyse trade executions, equity curves, and drawdown charts, complementing quantitative metrics with visual insights.

## 6 Work Schedule

In this section, we outline the timeline for developing the proposed implementation of the Forex trading system. The project is scheduled to run from June 2024 until February 2025. This timeline is designed to accommodate the development, testing, and evaluation phases of the project. As with any complex project, this schedule may undergo adjustments to address the challenges that arise during the development process.

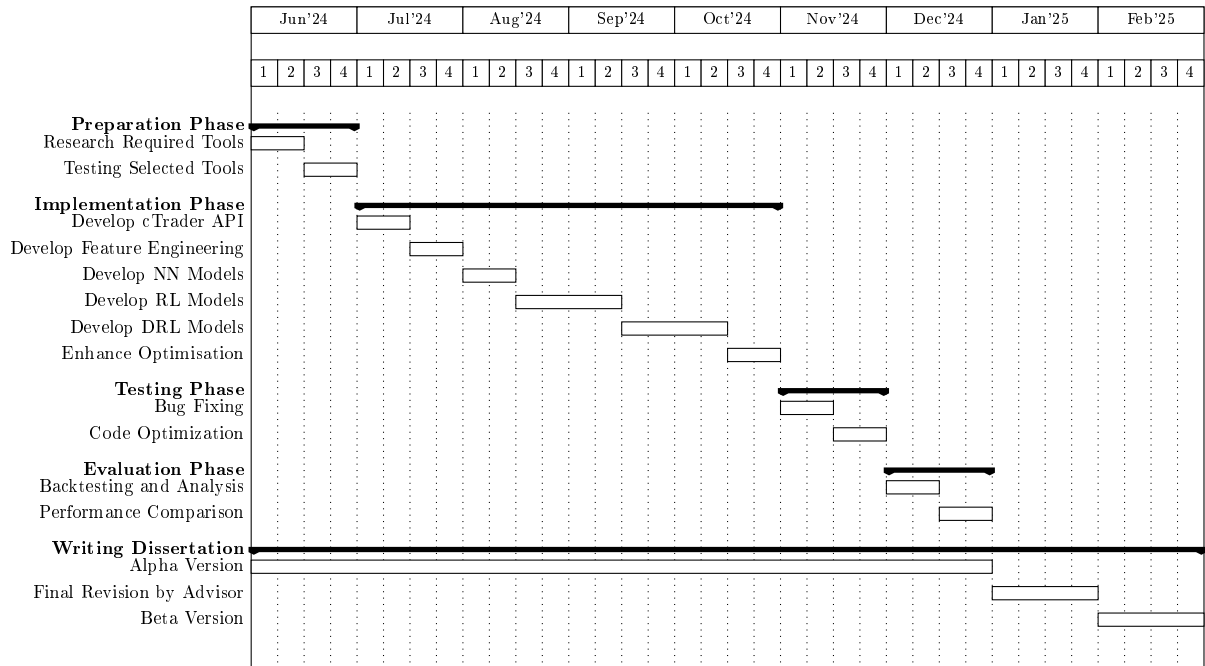


Figure 5: Planned Schedule

## 7 Conclusion

In summary, this research proposal aims to build an automated Forex trading system that uses Deep Deterministic Policy Gradient (DDPG) algorithm to generate trading signals. It intends to implement Recurrent Neural Networks (RNN), particularly Long-Short Term Memory (LSTM) networks, under the DRL model and incorporate sophisticated feature engineering and optimisation techniques, addressing significant gaps in the current literature. The proposed system is intended to integrate with the cTrader platform, enabling real-time data processing, online learning, and easy live deployment, along with an advanced evaluation framework, including visual backtesting and expert performance metrics. This research seeks to advance the state-of-the-art in automated Forex trading, providing a robust and scalable solution for algorithmic trading in complex and dynamic market environments.

## Bibliography

- [1] “Forex & Algorithmic Trading Community | cTrader Community.” [Online]. Available: <https://ctrader.com/>
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, “Playing Atari with Deep Reinforcement Learning,” *ArXiv*, Dec. 2013. [Online]. Available: <https://www.semanticscholar.org/paper/Playing-Atari-with-Deep-Reinforcement-Learning-Mnih-Kavukcuoglu/2319a491378867c7049b3da055c5df60e1671158>
- [3] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. [Online]. Available: <https://www.nature.com/articles/nature14236>
- [4] H. Van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-Learning,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, Mar. 2016, iSSN: 2374-3468, 2159-5399 Issue: 1 Journal Abbreviation: AAAI. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/10295>
- [5] T. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” *CoRR*, Sep. 2015. [Online]. Available: <https://www.semanticscholar.org/paper/Continuous-control-with-deep-reinforcement-learning-Lillicrap-Hunt/024006d4c2a89f7acacc6e4438d156525b60a98f>
- [6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, Nov. 2016, google-Books-ID: omivDQAAQBAJ.
- [7] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, Nov. 2018, google-Books-ID: sWV0DwAAQBAJ.
- [8] P. Brandimarte, *An Introduction to Financial Markets: A Quantitative Approach*. John Wiley & Sons, Feb. 2018, google-Books-ID: \_2tODwAAQBAJ.
- [9] B. Donnelly, *The Art of Currency Trading: A Professional’s Guide to the Foreign Exchange Market*. John Wiley & Sons, Jun. 2019, google-Books-ID: r8ybDwAAQBAJ.
- [10] R. Rhoads, *Candlestick Charting For Dummies*. John Wiley & Sons, Jun. 2022, google-Books-ID: 92l6EAAAQBAJ.
- [11] R. Teo, “Candlestick Pattern OHLC.” [Online]. Available: <https://www.tradingwithrayner.com/wp-content/uploads/2019/04/candlestick-pattern-ohlc.png>
- [12] M. A. Lim, *The Handbook of Technical Analysis + Test Bank: The Practitioner’s Comprehensive Guide to Technical Analysis*. John Wiley & Sons, Dec. 2015, google-Books-ID: N01\_BwAAQBAJ.
- [13] S. Jansen, *Machine Learning for Algorithmic Trading*. Packt Publishing Ltd, Jul. 2020, google-Books-ID: 4f30DwAAQBAJ.
- [14] E. P. Chan, *Quantitative Trading: How to Build Your Own Algorithmic Trading Business*. John Wiley & Sons, Jun. 2021, google-Books-ID: pWE0EAAAQBAJ.
- [15] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997, google-Books-ID: EoYBNgEACAAJ.
- [16] R. Pramoditha, “Human Neuron vs Artificial Neuron.” [Online]. Available: [https://miro.medium.com/v2/resize:fit:2902/format:webp/1\\*hkYlTODpjJgo32DoCOWN5w.png](https://miro.medium.com/v2/resize:fit:2902/format:webp/1*hkYlTODpjJgo32DoCOWN5w.png)

- [17] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. [Online]. Available: <https://doi.apa.org/doi/10.1037/h0042519>
- [18] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986. [Online]. Available: <https://www.nature.com/articles/323533a0>
- [19] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, Jan. 1989. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/0893608089900208>
- [20] B. F. Skinner, "Operant behavior," *American Psychologist*, vol. 18, no. 8, pp. 503–515, 1963, place: US Publisher: American Psychological Association.
- [21] B. Hambly, R. Xu, and H. Yang, "Recent advances in reinforcement learning in finance," *Mathematical Finance*, vol. 33, no. 3, pp. 437–503, Jul. 2023. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1111/mafi.12382>
- [22] T. G. Fischer, "Reinforcement learning in financial markets - a survey," *Friedrich-Alexander-Universität Erlangen-Nürnberg, Institute for Economics*, 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Reinforcement-learning-in-financial-markets-a-Fischer/922864ede84bc49be4ac676951278a9b568b6383>
- [23] T.-V. Pricope, "Deep Reinforcement Learning in Quantitative Algorithmic Trading: A Review," *ArXiv*, May 2021. [Online]. Available: <https://www.semanticscholar.org/paper/Deep-Reinforcement-Learning-in-Quantitative-A-Pricope/a6a7e7be402fc5f7703e5c74a7b2afa84613a429>
- [24] A. Alameer, H. Saleh, and K. Alshehri, "Reinforcement Learning in Quantitative Trading: A Survey," *TechRxiv*, Mar. 2022. [Online]. Available: <https://www.techrxiv.org/doi/full/10.36227/techrxiv.19303853.v1>
- [25] J. Carapuço, R. Neves, and N. Horta, "Reinforcement learning applied to Forex trading," *Applied Soft Computing*, vol. 73, pp. 783–794, Dec. 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1568494618305349>
- [26] C.-Y. Huang, "Financial Trading as a Game: A Deep Reinforcement Learning Approach," *ArXiv*, Jul. 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Financial-Trading-as-a-Game%3A-A-Deep-Reinforcement-Huang/a13a070940d48bce111f31ba9c0233884a17ae62>
- [27] F. Rundo, "Deep LSTM with Reinforcement Learning Layer for Financial Trend Prediction in FX High Frequency Trading Systems," *Applied Sciences*, vol. 9, no. 20, p. 4460, Oct. 2019. [Online]. Available: <https://www.mdpi.com/2076-3417/9/20/4460>
- [28] A. R. Azhikodan, A. G. K. Bhat, and M. V. Jadhav, "Stock Trading Bot Using Deep Reinforcement Learning," *Springer*, vol. 32, pp. 41–49, 2019, book Title: Innovations in Computer Science and Engineering Series Title: Lecture Notes in Networks and Systems. [Online]. Available: [http://link.springer.com/10.1007/978-981-10-8201-6\\_5](http://link.springer.com/10.1007/978-981-10-8201-6_5)
- [29] P. K. Gran, A. J. K. Holm, and S. Søgård, "A Deep Reinforcement Learning Approach to Stock Trading," *NTNU*, 2019. [Online]. Available: <https://www.semanticscholar.org/paper/A-Deep-Reinforcement-Learning-Approach-to-Stock-Gran-Holm/ef9f709e845d282cfb9371708d05d61580d1e10c>
- [30] M. Kong and J. So, "Empirical Analysis of Automated Stock Trading Using Deep Reinforcement Learning," *Applied Sciences*, vol. 13, no. 1, p. 633, Jan. 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/1/633>

- [31] Y. Li, W. Zheng, and Z. Zheng, "Deep Robust Reinforcement Learning for Practical Algorithmic Trading," *IEEE Access*, vol. 7, pp. 108 014–108 022, 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8786132/>
- [32] N. Majidi, M. Shamsi, and F. Marvasti, "Algorithmic Trading Using Continuous Action Space Deep Reinforcement Learning," *Expert Systems with Applications*, 2022, publisher: arXiv Version Number: 1. [Online]. Available: <https://arxiv.org/abs/2210.03469>
- [33] H. K. Sagiraju and S. Mogalla, "Application of multilayer perceptron to deep reinforcement learning for stock market trading and analysis," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 24, no. 3, p. 1759, Dec. 2021. [Online]. Available: <http://ijeecs.iaescore.com/index.php/IJECS/article/view/24816>
- [34] J. Sim and B. Kirk, "Generalized Deep Reinforcement Learning for Trading," *Journal of Student Research*, vol. 12, no. 1, Feb. 2023. [Online]. Available: <https://www.jsr.org/hs/index.php/path/article/view/4316>
- [35] T. Théate and D. Ernst, "An application of deep reinforcement learning to algorithmic trading," *Expert Systems with Applications*, vol. 173, p. 114632, Jul. 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417421000737>
- [36] X. Wu, H. Chen, J. Wang, L. Troiano, V. Loia, and H. Fujita, "Adaptive stock trading strategies with deep reinforcement learning methods," *Information Sciences*, vol. 538, pp. 142–158, Oct. 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025520304692>
- [37] Z. Xiong, X.-Y. Liu, S. Zhong, H. Yang, and A. Elwalid, "Practical Deep Reinforcement Learning Approach for Stock Trading," *ArXiv*, Nov. 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Practical-Deep-Reinforcement-Learning-Approach-for-Xiong-Liu/412fbb7ffa29aa6b2e172c1f545223baf18f1b6e>
- [38] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, "Deep reinforcement learning for automated stock trading: an ensemble strategy," *Proceedings of the First ACM International Conference on AI in Finance*, pp. 1–8, Oct. 2020, conference Name: ICAIF '20: ACM International Conference on AI in Finance ISBN: 9781450375849 Place: New York New York Publisher: ACM. [Online]. Available: <https://dl.acm.org/doi/10.1145/3383455.3422540>
- [39] N. Yousefi, "Deep Reinforcement Learning for Tehran Stock Trading," *Journal of Novel Engineering Science and Technology*, vol. 1, no. 02, pp. 37–42, Nov. 2022. [Online]. Available: <https://journal.iistr.org/index.php/JNEST/article/view/171>
- [40] L. Chen and Q. Gao, "Application of Deep Reinforcement Learning on Automated Stock Trading," *2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS)*, pp. 29–33, Oct. 2019, conference Name: 2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS) ISBN: 9781728109459 Place: Beijing, China Publisher: IEEE. [Online]. Available: <https://ieeexplore.ieee.org/document/9040728/>
- [41] Z. Jia, Q. Gao, and X. Peng, "LSTM-DDPG for Trading with Variable Positions," *Sensors*, vol. 21, no. 19, p. 6571, Sep. 2021. [Online]. Available: <https://www.mdpi.com/1424-8220/21/19/6571>
- [42] Z. Zhang, S. Zohren, and S. Roberts, "Deep Reinforcement Learning for Trading," *The Journal of Financial Data Science*, vol. 2, pp. 25–40, Apr. 2020, iSSN: 2640-3943 Issue: 2 Journal Abbreviation: JFDS. [Online]. Available: <http://pm-research.com/lookup/doi/10.3905/jfds.2020.1.030>
- [43] "MetaTrader 5 Trading Platform for Forex, Stocks, Futures." [Online]. Available: <https://www.metatrader5.com>
- [44] "CFD & Forex Trading | Stocks & Commodities | IC Markets EU." [Online]. Available: <https://www.icmarkets.eu>